

REDES NEURONALES CON FRAMEWORKS

Potencia, riesgo y responsabilidad técnica

BLOQUE 0 · ENFOQUE GENERAL DE LA SESIÓN

Hasta ahora has trabajado con redes neuronales en un entorno **deliberadamente controlado**:

- arquitecturas simples,
- pocas decisiones explícitas,
- límites claros,
- feedback inmediato.

Esto no ha sido una limitación del curso.

Ha sido una **decisión pedagógica consciente**.

Durante las sesiones anteriores no se ha buscado “hacer redes neuronales”, sino **entrenar criterio técnico**.

En esta sesión das un salto cualitativo:

pasas de operar una red
a **construirla explícitamente**.

Ese salto **no es solo técnico**.

Es un salto de **responsabilidad profesional**.

A partir de ahora, muchas decisiones que antes estaban ocultas pasan a ser **decisiones tuyas**.

BLOQUE 1 · POR QUÉ NECESITAMOS FRAMEWORKS

El límite de **MLPClassifier**

MLPClassifier ha sido utilizado como herramienta de aprendizaje porque:

- protege de decisiones prematuras,
- permite comparar modelos,
- facilita la evaluación,
- y expone bien el sobreajuste.

Pero tiene límites claros:

- no permite arquitecturas arbitrarias,
- no expone el flujo interno de la red,
- no escala a problemas reales,
- no ofrece control fino del entrenamiento.

Cuando el problema crece, necesitas un **framework de deep learning**.

No para "hacer más cosas".

Sino para **hacerlas de forma explícita**.

BLOQUE 2 · ENTRA KERAS (Y QUÉ CAMBIA REALMENTE)

Se introduce **TensorFlow / Keras** con una advertencia esencial:

Keras no te da mejores modelos.

Te da **más palancas**.

Y cada palanca mal utilizada:

- aumenta el riesgo de sobreajuste,
- oculta errores de diseño,
- produce modelos frágiles.

Primer contacto mínimo y consciente

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(10, activation="relu", input_shape=(X_train.shape[1],)),
    Dense(1, activation="sigmoid")
])
```

Aquí **no se busca optimizar nada**.

Solo observar una diferencia clave:

ahora la arquitectura ya no está implícita.

Está **declarada por ti**.

Las redes neuronales no son peligrosas por ser potentes.

Son peligrosas cuando se usan sin diagnóstico.

BLOQUE 3 · CATÁLOGO DE ERRORES SILENCIOSOS

Cuando el código funciona... pero el modelo no

En Keras ocurre algo nuevo y peligroso:

el código puede ejecutarse perfectamente

y el modelo estar **técnicamente muerto**.

Aquí se introduce la **depuración lógica**, no sintáctica.

Error 1 · La trampa de la activación

Usar una activación incorrecta en la salida:

```
Dense(1, activation="relu")
```

en un problema de probabilidad binaria.

El código no falla.

El entrenamiento ocurre.

Pero el modelo **no representa probabilidades válidas**.

Resultado: métricas incoherentes y decisiones erróneas.

Error 2 · La escala importa (y mucho)

Entrenar en Keras **sin escalar los datos**:

- gradientes inestables,
- convergencia errática,
- aprendizaje extremadamente lento o inexistente.

Mientras que algunos modelos clásicos pueden "aguantar",

Keras es **implacable** con este error.

El framework no te protege.

Te exige responsabilidad.

Cuando el error no es visible en el código,

solo queda una defensa: observar el entrenamiento.

El "Check de Cordura" (Sanity Check)

Antes de mirar **accuracy**, **precision** o cualquier métrica final,
un profesional realiza un control mínimo de coherencia.

Este control no evalúa si el modelo es bueno.

Evalúa si el modelo **tiene derecho a ser evaluado**.

1. ¿La pérdida inicial tiene sentido?

En un problema de **clasificación binaria** con **entropía cruzada**:

- si los pesos son aleatorios,
- y el modelo no ha aprendido nada,

la pérdida inicial debería estar cerca de:

| ≈ 0.69

Si la pérdida inicial es:

- extremadamente baja → posible fuga de datos,
- extremadamente alta → error de escala, activación o planteamiento.

Un número "raro" aquí invalida cualquier métrica posterior.

2. ¿La salida tiene el rango correcto?

Antes de interpretar predicciones, verifica:

- si la activación final es `sigmoid`,
- las salidas deben estar **entre 0 y 1**.

Valores fuera de ese rango indican:

- activación incorrecta,
- arquitectura mal definida,
- o interpretación errónea del output.

| Si el rango no es correcto,

| **el modelo no está resolviendo el problema que crees.**

Idea clave del Bloque 3

| Cuando el error no es visible en el código,
| solo queda una defensa: **comprobar la coherencia del modelo**
| antes de confiar en cualquier métrica.

Este control es rápido, barato y decisivo.

Saltárselo es uno de los errores profesionales más comunes
al trabajar con frameworks de deep learning.

(Transición natural al Bloque 4)

A partir de aquí, el siguiente paso ya no es "probar más capas", sino **observar el entrenamiento**.
Y eso exige trazabilidad.

BLOQUE 4 · ENTRENAMIENTO Y TRAZABILIDAD

Del gráfico simple al "dashboard de vuelo"

Hasta ahora has visto curvas de pérdida con `plt.plot()`.

Eso es suficiente para aprender.

No para trabajar profesionalmente.

Un profesional **no se queda mirando la pantalla** mientras el modelo entrena.

| Programa al modelo para que se cuide solo.

Introducción conceptual a *Callbacks*

- `EarlyStopping`: detener el entrenamiento cuando deja de mejorar.
- `ModelCheckpoint`: guardar el mejor modelo automáticamente.

```
from tensorflow.keras.callbacks import EarlyStopping

callback = EarlyStopping(
    monitor="val_loss",
    patience=5,
    restore_best_weights=True
)
```

Idea clave:

| El entrenamiento también se diseña.
| No se observa pasivamente.

Esto refuerza la noción de **trazabilidad y control**, introducida en la Sesión 9.

BLOQUE 5 · MISMO PROBLEMA, MISMAS REGLAS

Regla de oro (enunciada explícitamente):

Cambiar de herramienta
no cambia las reglas del juego.

Se comparan:

- Regresión logística,
- `MLPClassifier`,
- Keras.

Siempre con:

- mismo dataset,
- misma división,
- misma métrica,
- mismo baseline.

Y aparece una lección clave:

A veces Keras no gana.
Y cuando no gana, tampoco fracasa: simplemente no era necesario.

Y eso **está bien**.

BLOQUE 6 · EL ÁRBOL DE DECISIÓN DEL ARQUITECTO

¿Scikit-Learn o Keras?

Elegir un framework **es una decisión de arquitectura**, no de moda.

Criterio	Scikit-Learn	Zona intermedia (criterio)	Keras / Framework
Volumen de datos	< 10.000 filas	10.000 – 100.000 filas	> 100.000 filas

Criterio	Scikit-Learn	Zona intermedia (criterio)	Keras / Framework
Naturaleza del dato	Tabular, estructurado	Tabular complejo / mixto	Imagen, audio, texto denso
Relación entrada-salida	Mayormente lineal	No lineal moderada	Altamente no lineal
Tiempo de entrega	Baseline rápido	Iteración limitada pero posible	Iteración larga
Hardware	CPU	CPU potente / GPU opcional	GPU / TPU
Interpretabilidad	Alta	Media (negociable)	Baja
Riesgo de sobreajuste	Bajo	Alto si no se controla	Muy alto

Idea clave:

Usar Keras cuando no lo necesitas
es introducir complejidad innecesaria.

BLOQUE 7 · INFERENCIA: EL MODELO EN EL MUNDO REAL

Entrenar un modelo **no es el final**.

En producción, el modelo:

- se guarda,
- se carga,
- se usa para inferir.

Comparación conceptual:

- un modelo `.pkl` de Scikit-Learn
- frente a un modelo de Keras.

Diferencias reales:

- tamaño en disco,
- consumo de memoria,
- latencia de inferencia.

Mensaje clave:

La potencia de Keras tiene un coste.

¿Tu sistema puede permitírselo?

BLOQUE 8 · CIERRE DEL CURSO

Contrato de responsabilidad del desarrollador

Antes de cerrar el curso, se establece un compromiso profesional.

Contrato (no escrito, pero vinculante)

1. No usaré una red neuronal

si una regresión logística ofrece el mismo rendimiento.

2. No añadiré capas "por si acaso"

sin mirar la curva de validación.

3. Soy responsable de cada hiperparámetro

que dejo por defecto.

Este no es un contrato académico.

Es un **compromiso de oficio**.

CONCLUSIÓN FINAL

Has recorrido todo el arco del curso:

- diseñaste variables,
- evaluaste modelos,
- entendiste métricas,

- controlaste el sobreajuste,
- aprendiste cuándo delegar representación,
- y ahora sabes **cuándo usar herramientas potentes.**

La diferencia entre un modelo académico
y un sistema profesional
no es la librería.

Es el **criterio con el que se usa.**